Vietnam National University - Ho Chi Minh city
University of Technology
Faculty of Computer Science and Engineering

CO3067 - Parallel Computing

Parallelizing Strassen's Algorithm for Matrix Multiplication

Students:
Nguyễn Văn An - 2352013
Lê Hữu Triều - 2353219

Instructor: Diệp Thanh Đăng

Ho Chi Minh city, December, 2025

Contents

1 Introduction **2**

2 Experimental Setup **2**

3 Our proposed naive matrix multiplication algorithm **3**
3.1 The naive matrix multiplication algorithm . . . 3
3.2 Strategies to improve the naive algorithm . . . 3
3.2.1 Loop Tiling (Blocking) . . . 3
3.2.2 Local Accumulation . . . 3
3.2.3 Compiler-assisted Auto-vectorization . . . 4

4 Strassen's Algorithm **5**
4.1 Overview . . . 5
4.2 The Strassen-Winograd Variant . . . 5
4.3 Memory-Efficient Sequential Implementation . . . 6

5 Parallel Implementation with OpenMP Tasks **7**
5.1 Task Division Strategy . . . 7
5.1.1 Phase 1: Global Pre-computation . . . 7
5.1.2 Phase 2: Parallel Task Execution . . . 7
5.1.3 Phase 3: Reconstruction . . . 8
5.2 Hybrid Calculation Strategy . . . 8

6 Parallel Implementation with MPI **9**
6.1 Process Model and Design Rationale . . . 9
6.2 Data Management . . . 9
6.3 Intergrating OpenMP . . . 10

7 Results **11**

8 Discussion **12**
8.1 Impact of Sequential Optimizations . . . 12
8.2 Scaling Analysis . . . 12
8.3 Performance Bottlenecks in the Cluster Implementation . . . 12
8.4 Further improvements . . . 13

9 Conclusion **13**

1 Introduction

Matrix multiplication is a fundamental operation in high-performance computing, serving as a cornerstone for scientific simulations, machine learning, and data analysis. While the standard naive matrix multiplication algorithm operates with a time complexity of $O(n^3)$, this computational cost becomes prohibitive for large-scale matrices. In 1969, Volker Strassen introduced the first sub-cubic algorithm, reducing the complexity to approximately $O(n^{2.81})$ by trading expensive multiplication operations for cheaper additions.

This report investigates the implementation and optimization of Strassen's algorithm using various parallel computing techniques. We explore three primary approaches:

1. Optimized Sequential: Enhancing cache locality through loop tiling and vectorization.
2. Shared-Memory Parallelism: Utilizing OpenMP tasks to parallelize recursive calls on multi-core processors.
3. Distributed-Memory Parallelism: Implementing a Hybrid MPI+OpenMP model to scale computations across a computer cluster.

The objective is to analyze the performance characteristics of these methods, determine the impact of hybrid parallelization, and compare our results against industry-standard libraries like OpenBLAS and NumPy.

2 Experimental Setup

The algorithms discussed in this report were implemented in C++23. The performance benchmarks were conducted on a machine with the following specifications:

- **Compilers and Environments:**
  - Sequential (Naive): Compiled with `g++`.
  - OpenMP: Compiled with `g++` using the `-fopenmp` flag.
  - MPI / Hybrid: Compiled with `mpic++` and executed via the `mpirun` launcher.
- **Global Compilation Flags:** `-Ofast -march=native` (applied to all configurations).

The compilation flags serve the following specific purposes:

- `-Ofast`: Enables aggressive optimizations, including `-O3` (high-level optimization) and `-ffast-math` (allows reordering of floating-point operations).
- `-march=native`: Optimizes the binary for the host machine's specific instruction set (enabling AVX/AVX2/AVX-512 extensions).
- `-fopenmp`: Enables the OpenMP runtime for multi-threading.

All time measurements reported are the average of multiple runs to minimize noise from background system processes.

3 Our proposed naive matrix multiplication algorithm

3.1 The naive matrix multiplication algorithm

The naive matrix multiplication algorithm involves three nested loops to compute the product of two matrices. For matrices A of size $m \times n$ and B of size $n \times p$, the algorithm iterates through each row of A and each column of B, calculating the dot product to fill the resulting matrix C of size $m \times p$. The time complexity of this algorithm is $O(m \cdot n \cdot p)$ while the space complexity is $O(m \cdot p)$ for storing the result.

```
void naive_matmul(const Matrix_t& A, const Matrix_t& B, Matrix_t& C) {
    size_t size = std::size(A);
    for (size_t i = 0; i < size; i++) {
        for (size_t j = 0; j < size; j++) {
            for (size_t k = 0; k < size; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}
```

Listing 1: Naive Matrix Multiplication Algorithm

3.2 Strategies to improve the naive algorithm

To mitigate the performance bottlenecks of the naive approach, specifically the poor cache locality and lack of parallelism, we implemented an optimized matrix multiplication routine. This implementation integrates two primary optimization strategies: Loop Tiling and Local Accumulation.

3.2.1 Loop Tiling (Blocking)

The primary optimization applied is Loop Tiling. Instead of iterating over the entire matrix dimensions linearly, the algorithm divides the matrices into smaller sub-blocks of size $BS \times BS$ (defined as `blockSize`). This strategy ensures that the working set of data fits within the CPU's high-speed cache (L1/L2), significantly reducing cache misses. As shown in Algorithm 1, the implementation iterates through these chunks using outer loops (ii, kk, jj) before processing the internal elements.

3.2.2 Local Accumulation

To further reduce memory traffic to the global result matrix C, the algorithm utilizes a local accumulator. A temporary buffer, `localC`, is allocated on the stack to store intermediate results for the current block. The results are accumulated in this local buffer and strictly written back to the main memory matrix C only after the block computation is complete. This minimizes the frequency of expensive write operations to the global memory.

3.2.3 Compiler-assisted Auto-vectorization

In this implementation, we rely on the compiler's auto-vectorization capabilities, enabled by the `-Ofast` and `-march=native` flags.

- **`-Ofast`**: This flag enables `-O3` optimizations (aggressive loop unrolling and inlining) and `-ffast-math`. The latter is particularly crucial for matrix multiplication because it relaxes IEEE 754 floating-point strictness, allowing the compiler to reassociate operations (e.g. $(a+b)+c$ vs $a+(b+c)$). This flexibility allows the compiler to vectorize accumulation loops that would otherwise be strictly serialized.
- **`-march=native`**: This instructs the compiler to generate machine-specific assembly code for the host CPU. By detecting support for advanced instruction sets like AVX2 or AVX-512, the compiler can generate vector instructions that process 8 or 16 single-precision floats per cycle, rather than the default generic instructions.

Combined with Loop Tiling (which ensures data resides in L1 cache), these flags allow the compiler to automatically generate highly efficient vectorized assembly for the inner accumulation loops.

Algorithm 1 Improved Matrix Multiplication (Tiling + Accumulation)

Require: Matrices A, B, Result Matrix C, Block Size BS
Ensure: $C \mathrel{+}= A \times B$

```
for ii ← 0 to N step BS do                                  ▷ Outer Row
    for kk ← 0 to N step BS do                              ▷ Outer Common
        for jj ← 0 to N step BS do                          ▷ Outer Col
            localC[BS][BS] ← 0                              ▷ Init local accumulator
            for i ← ii to min(ii + BS, N) do
                for k ← kk to min(kk + BS, N) do
                    val ← A(i, k)
                    for j ← jj to min(jj + BS, N) do
                        localC[i − ii][j − jj] += val × B(k, j)
                    end for
                end for
            end for
                                                            ▷ Write back to global memory
            for i ← ii to min(ii + BS, N) do
                for j ← jj to min(jj + BS, N) do
                    C(i, j) += localC[i − ii][j − jj]
                end for
            end for
        end for
    end for
end for
```

4 Strassen's Algorithm

4.1 Overview

Strassen's Algorithm, introduced by Volker Strassen in 1969, was the first algorithm to prove that matrix multiplication can be performed in sub-cubic time, specifically $O(n^{\log_2 7}) \approx O(n^{2.81})$. The core insight of Strassen's approach is to trade expensive multiplication operations for cheaper addition operations. By recursively partitioning the matrices into quadrants, the standard divide-and-conquer approach requires 8 multiplications and 4 additions per step. Strassen's method reduces this to 7 multiplications at the cost of increasing the number of additions.

4.2 The Strassen-Winograd Variant

Our implementation uses the Strassen-Winograd variant of the algorithm. This specific formulation reduces the number of matrix additions and subtractions from 18 to 15 comparing to the standard implementation, improving the constant factor of the complexity while maintaining the asymptotic $O(n^{2.81})$ behavior.

The algorithm operates on input matrices A and B by partitioning them into four $\frac{n}{2} \times \frac{n}{2}$ sub-blocks:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \tag{1}$$

The computation proceeds in three distinct phases:

Phase 1 (8 Additions): First, we compute auxiliary matrices S and T via linear combinations of the blocks of A and B.

$$\begin{aligned} S_1 &= A_{21} + A_{22} & T_1 &= B_{12} - B_{11} \\ S_2 &= S_1 - A_{11} & T_2 &= B_{22} - T_1 \\ S_3 &= A_{11} - A_{21} & T_3 &= B_{22} - B_{12} \\ S_4 &= A_{12} - S_2 & T_4 &= T_2 - B_{21} \end{aligned}$$

Phase 2 (7 Multiplications): We then perform seven recursive matrix multiplications to obtain intermediate products P_1 through P_7. In our implementation, if the matrix size n drops below a tuned threshold (n_{base}), these multiplications are performed using the optimized tiling algorithm described in Section 3.

$$\begin{aligned} P_1 &= A_{11} \times B_{11} & P_5 &= S_1 \times T_1 \\ P_2 &= A_{12} \times B_{21} & P_6 &= S_2 \times T_2 \\ P_3 &= S_4 \times B_{22} & P_7 &= S_3 \times T_3 \\ P_4 &= A_{22} \times T_4 \end{aligned}$$

Phase 3 (7 Additions): Finally, the four quadrants of the result matrix C are computed. To conserve memory, our implementation reuses the storage allocated for S and T to store these

final sums (U).

$$U_1 = P_1 + P_2 \qquad C_{11} = U_1$$
$$U_2 = P_1 + P_6 \qquad C_{12} = U_5 = U_4 + P_3$$
$$U_3 = U_2 + P_7 \qquad C_{21} = U_6 = U_3 - P_4$$
$$U_4 = U_2 + P_5 \qquad C_{22} = U_7 = U_3 + P_5$$

4.3 Memory-Efficient Sequential Implementation

For the sequential component of our hybrid algorithm, we utilized the memory-efficient schedule proposed by Boyer et al. (see [1]). Standard Strassen implementations typically require substantial auxiliary memory for the 7 intermediate products and various block sums.

To minimize this overhead, our implementation adopts a specific execution schedule that requires only two temporary block matrices (X and Y) of size $\frac{n}{2} \times \frac{n}{2}$. As demonstrated in Table 1, we reuse the memory of the result matrix C to store intermediate products (P_i) and overwrite the temporaries (X, Y) as soon as their values are consumed.

Table 1: Sequential Strassen-Winograd Schedule (Two temporaries)

Step	Operation (Mathematical)	Code Operation	Storage
1	$S_3 = A_{11} - A_{21}$	`sub(A11, A21, X)`	X
2	$T_3 = B_{22} - B_{12}$	`sub(B22, B12, Y)`	Y
3	$P_7 = S_3 \times T_3$	`_strassen(X, Y, C21)`	C_{21}
4	$S_1 = A_{21} + A_{22}$	`add(A21, A22, X)`	X
5	$T_1 = B_{12} - B_{11}$	`sub(B12, B11, Y)`	Y
6	$P_5 = S_1 \times T_1$	`_strassen(X, Y, C22)`	C_{22}
7	$S_2 = S_1 - A_{11}$	`X.sub(A11)`	X
8	$T_2 = B_{22} - T_1$	`sub(B22, Y, Y)`	Y
9	$P_6 = S_2 \times T_2$	`_strassen(X, Y, C12)`	C_{12}
10	$S_4 = A_{12} - S_2$	`sub(A12, X, X)`	X
11	$P_3 = S_4 \times B_{22}$	`_strassen(X, B22, C11)`	C_{11}
12	$P_1 = A_{11} \times B_{11}$	`_strassen(A11, B11, X)`	X
13	$U_2 = P_1 + P_6$	`C12.add(X)`	C_{12}
14	$U_3 = U_2 + P_7$	`C21.add(C12)`	C_{21}
15	$U_4 = U_2 + P_5$	`C12.add(C22)`	C_{12}
16	$U_7 = U_3 + P_5$	`C22.add(C21)`	C_{22}
17	$U_5 = U_4 + P_3$	`C12.add(C11)`	C_{12}
18	$T_4 = T_2 - B_{21}$	`Y.sub(B21)`	Y
19	$P_4 = A_{22} \times T_4$	`_strassen(A22, Y, C11)`	C_{11}
20	$U_6 = U_3 - P_4$	`C21.sub(C11)`	C_{21}
21	$P_2 = A_{12} \times B_{21}$	`_strassen(A12, B21, C11)`	C_{11}
22	$U_1 = P_1 + P_2$	`C11.add(X)`	C_{11}

This schedule effectively performs all 22 block operations (15 additions/subtractions and 7 multiplications) while keeping auxiliary memory usage constant at $2 \times (n/2)^2$ per recursion level, avoiding the memory explosion typical of naive Strassen implementations.

5 Parallel Implementation with OpenMP Tasks

To leverage multi-core architectures, we implemented a parallel version of Strassen's algorithm using OpenMP tasks. Unlike the sequential version, this implementation cannot share restricted temporary memory buffers without introducing race conditions. Therefore, at each parallel recursion level, we explicitly allocate a larger memory buffer to store all necessary intermediate matrices.

5.1 Task Division Strategy

The execution flow at each recursive step is divided into three distinct phases: global pre-computation, parallel task execution, and post-computation reconstruction.

5.1.1 Phase 1: Global Pre-computation

Before spawning parallel tasks, the main thread computes four shared intermediate matrices (S_1, S_2, S_4, S_5) that are required by multiple subsequent tasks. These are stored in the allocated memory buffer.

Table 2: Global Pre-computation Steps

Step	Operation (Math)	Code Operation
1	$S_1 = A_{21} + A_{22}$	`add(A21, A22, S1)`
2	$S_2 = S_1 - A_{11}$	`sub(S1, A11, S2)`
3	$T_1 = B_12 - B_{11}$	`sub(B12, B11, T1)`
4	$T_2 = B_{22} - T_1$	`sub(B22, T1, T2)`

5.1.2 Phase 2: Parallel Task Execution

The seven recursive matrix multiplications are distributed across independent OpenMP tasks. To maximize parallelism and data locality, specific tasks perform their own local pre-calculations (computing unique S matrices) immediately before the multiplication. This design allows threads to compute their inputs independently without waiting for the main thread.

Table 3: Parallel Tasks and Local Calculations

Task	Operations (Math)	Code Operations
1	$P_1 = A_{11} \times B_{11}$	`_strassenOMP(A11, B11, P1)`
2	$P_2 = A_{12} \times B_{21}$	`_strassenOMP(A12, B21, C11)`
3	$S_4 = A_{12} - S_2$ $C_{12} = S_4 \times B_{22}$	`sub(A12, S2, S4)` `_strassenOMP(S4, B22, C12)`
4	$T_4 = T_2 - B_{21}$ $C_{21} = A_{22} \times T_4$	`sub(T2, B21, T4)` `_strassenOMP(A22, T4, C21)`
5	$C_{22} = S_1 \times T_1$	`_strassenOMP(S1, T1, C22)`
6	$U_2 = S_2 \times T_2$	`_strassenOMP(S2, T2, U2)`

7	$S_3 = A_{11} - A_{21}$
	$T_3 = B_{22} - B_{12}$
	$U_3 = S_3 \times T_3$
		`sub(A11, A21, S3)`
		`sub(B22, B12, T3)`
		`_strassenOMP(S3, T3, U3)`

Following the creation of these tasks, a synchronization barrier (`#pragma omp taskwait`) ensures that all intermediate products (V, U, T) and partial results (C_{ij}) are fully computed before the algorithm proceeds to the next phase.

5.1.3 Phase 3: Reconstruction

Once the tasks are synchronized, the main thread reconstructs the final quadrants of the result matrix C by linearly combining the intermediate products stored in the buffers and the partial results in C.

Table 4: Result Reconstruction Steps

Step	Operation (Math)	Code Operation
1	$C_{11} \mathrel{+}= P_1$	`C11.add(P1)`
2	$U_2 \mathrel{+}= P_1$	`U2.add(P1)`
3	$U_3 \mathrel{+}= U_2$	`U3.add(U2)`
4	$C_{12} \mathrel{+}= U_2$	`C12.add(U2)`
5	$C_{12} \mathrel{+}= C_{22}$	`C12.add(C22)`
6	$C_{21} := U_3 - C_{21}$	`sub(U3, C21, C21)`
7	$C_{22} \mathrel{+}= U_3$	`C22.add(U3)`

5.2 Hybrid Calculation Strategy

Our implementation employs a hybrid approach that transitions between three different algorithms: Parallel Strassen, Sequential Strassen, and Improved Matrix Multiplication based on the problem size and recursion depth.

1. Transition to Sequential Strassen (`depth` > `maxParallelDepth`): Creating OpenMP tasks incurs scheduling overhead. When the recursion depth exceeds a tuned limit

(`maxParallelDepth`), the algorithm stops spawning new tasks and switches to the memory-efficient Sequential Strassen implementation described in Section 4.3. This limits the total number of tasks and reduces memory pressure at deeper levels.

2. Transition to Improved Matrix Multiplication ($n <$ `threshold`): For smaller matrices, the overhead of Strassen's recursion (memory allocation and extra additions) outweighs the benefit of reducing multiplications. When the matrix dimension n drops below a specific threshold (e.g., $n = 480$), the algorithm switches to the Improved Matrix Multiplication (tiled and vectorized) described in Section 3.

6 Parallel Implementation with MPI

To extend parallelism beyond a single shared-memory node, we implemented a distributed-memory version of Strassen's algorithm using the Message Passing Interface (MPI). Unlike the OpenMP task-based approach, which relies on shared address space and implicit synchronization, the MPI implementation explicitly distributes computation and data across multiple processes, each with its own private memory.

6.1 Process Model and Design Rationale

Our model follows a master–worker design in which a master process performs pre-computation and data partitioning, distributes the main parallelizable computation tasks to worker processes, and finally reconstructs the result. Distributing the seven matrix multiplications at each Strassen recursion step across MPI processes was considered, as our OpenMP implementation follows a similar workflow. However, implementing such a model using MPI would require fine-grained communication between processes and dynamic load balancing. These concerns belong to the broader study of distributed recursive algorithms and hierarchical scheduling, which is beyond the scope of this assignment.

Instead, we restrict task distribution to the first recursion level of the Strassen algorithm. This design represents a deliberate tradeoff: while limiting distribution to a single level constrains the maximum degree of parallelism to seven worker processes, it significantly reduces communication overhead and avoids the complexity of managing recursive inter-process dependencies. At this level, the master process precomputes and packs the input matrices for the seven sub-Strassen calls, distributes these inputs to exactly seven worker processes, and reconstructs the output matrix upon completion. Each worker process computes its assigned subproblem independently using its own local resources, while all deeper levels of recursion are executed locally to preserve computational efficiency.

6.2 Data Management

At level 1, the execution flow is relatively the same as of OpenMP implementation, with a few notes on how matrix data is distributed. Specifically, we want to allocate extra memory so that data passing between master-worker can be done in the least amount of API calls, because message passing between nodes is generally more costly than memory allocation in a single node. The master process does all the additions/subtractions before the seven multiplication,

packs the input matrices into contigous memory blocks, and send the blocks to worker processes. When receiving output sub-matrices from workers, the master process can resuse the allocated blocks to hold the result.

Algorithm 2 Data Management in Master Process

```
Compute S_1, S_2, S_3, S_4, T_1, T_2, T_3, T_4

data[0] ← pack(A_11, B_11)
data[1] ← pack(A_12, B_21)
data[2] ← pack(S_4, B_22)
data[3] ← pack(A_22, T_4)
data[4] ← pack(S_1, T_1)
data[5] ← pack(S_2, T_2)
data[6] ← pack(S_3, T_3)

for i ← 0 to 6 do
    sendData(data[i], worker[i])
end for

for i ← 0 to 6 do
    recvData(data[i], worker[i])
end for

P_1 ← data[0]
C_11 ← data[1]
C_12 ← data[2]
C_21 ← data[3]
C_22 ← data[4]
U_2 ← data[5]
U_3 ← data[6]

Compute C
```

Algorithm 3 Data Management in Worker Process

```
Allocate memory for matrix of size (2n × n)
recvData(allocated_mem, master)
(A, B) ← unpack(allocated_mem)
Strassen(A, B, C)
sendData(C, master)
```

6.3 Intergrating OpenMP

Although MPI is used to distribute computation across processes, each MPI process may execute on a multi-core node. In this case, the local sequential Strassen computation can be replaced with the OpenMP-based parallel Strassen implementation without modifying the

MPI communication structure. This hybrid MPI+OpenMP approach exploits coarse-grained parallelism across nodes via MPI while leveraging shared-memory parallelism within each node using OpenMP, improving overall resource utilization with minimal additional complexity.

7 Results

Below is our benchmark of the implemented algorithms.

Our MPI related implementations were tested on the provided computer cluster, while our sequential as well as OpenMP implementations were tested on 2 devices:

- Laptop A: AMD Ryzen 5 CPU, ran with 6 threads. Optimal configuration for this device was `blockSize = 128` and `threshold = 512`.
- Laptop B: 11th Gen Intel CPU, ran with 4 threads. Optimal configuration for this device was `blockSize = 480` and `threshold = 480`.

We ran on matrix sizes of 2000, 4000, 6000, 8000, and 10000. Each test was run 3 times and the average time was recorded. "TLE" indicates that the execution time exceeded our time limit of 10 minutes.

As part of the requirements, we also benchmarked the matrix multiplication from well-known libraries. Specifically, Laptop A used OpenBLAS while Laptop B used NumPy's built-in function.

Size	**Time (ms)**				
	Naive	**Improved**	**Strassen serial**	**Strassen OMP**	**OpenBLAS**
2,000	6744.61	342.97	354.29	108.52	42.26
4,000	136691.00	2880.57	2492.40	831.70	222.87
6,000	552019.00	10092.24	7720.80	2301.73	705.77
8,000	TLE	21924.07	16717.17	4976.20	1826.88
10,000	TLE	43913.83	32277.63	9914.28	3733.95

Table 5: Matrix Multiplication Execution Times on Laptop A

Size	**Time (ms)**				
	Naive	**Improved**	**Strassen Serial**	**Strassen OMP**	**NumPy**
2,000	11954.50	518.20	364.37	121.68	89.38
4,000	227965.00	4069.99	2918.12	851.77	717.76
6,000	TLE	10960.60	12125.10	2887.89	2436.79
8,000	TLE	25073.40	20859.70	6071.07	5886.11
10,000	TLE	56373.90	37148.30	12667.10	11834.79

Table 6: Matrix Multiplication Execution Times on Laptop B

Below is the runtime of our implementations on the provided computer cluster. Note that the MPI and the Hybrid implementations was run with 8 processes (i.e. `-np 8`). For comparison, we also ran the built-in NumPy function on 1 node:

Size	Time (ms)		
	Strassen MPI	**Strassen Hybrid**	**NumPy**
2,000	1956.90	911.14	589.33
4,000	11193.70	3878.87	2896.85
6,000	35789.20	10380.90	9803.70
8,000	77665.80	21872.30	21885.22
10,000	52110.20	19652.00	42018.48

Table 7: MPI and Hybrid Execution Times

8 Discussion

8.1 Impact of Sequential Optimizations

The experimental results highlight the critical role of memory hierarchy optimization. On Laptop A, the improved sequential algorithm (utilizing tiling and local accumulation) achieved a speedup of approximately 19.7× over the naive implementation for $N = 2000$ (342.97 ms vs 6744.61 ms). This confirms that mitigating cache misses and leveraging compiler auto-vectorization are prerequisites for high performance before algorithmic changes are even applied.

8.2 Scaling Analysis

The scaling behaviors of the implementations align with their theoretical complexities but reveal practical crossover points:

- Naive vs. Improved: Both scale at $O(n^3)$, but the Improved version has a significantly smaller constant factor due to better cache usage.
- Strassen vs. Improved: On Laptop A, the Improved algorithm was actually faster than Strassen Serial at $N = 2000$ (342.97 ms vs 354.29 ms). However, the crossover occurs at $N = 4000$, where Strassen Serial becomes faster than the Improved version (2492.40 vs 2880.57 ms), validating Strassen's asymptotic advantage for larger inputs.
- OpenMP Scaling: The Strassen OMP implementation demonstrated reasonable scaling on shared memory, maintaining the lowest execution times across all our tests on Laptop A (9914.28 ms for $N = 10000$).

8.3 Performance Bottlenecks in the Cluster Implementation

Contrary to the expectation that more resources equals faster speed, our Cluster implementations (MPI and Hybrid) were slower than the single-node OpenMP implementation on Laptop A. For $N = 10000$, Strassen OMP took ≈ 9.9s, whereas Strassen Hybrid took ≈ 19.6s and pure Strassen MPI took ≈ 52.1s.

This counter-intuitive result is attributed to two factors:

1. Strassen's algorithm requires exchanging large sub-matrices. In a distributed environment, the latency and bandwidth cost of sending these matrices over the network significantly

outweighs the computational time saved, especially when the matrix size is not massive enough to justify the transfer cost.

2. As detailed in Section 6.1, our MPI design only distributes the first level of recursion, limiting the system to exactly 7 worker processes. In contrast, the single-node OpenMP implementation can dynamically spawn tasks for multiple recursion levels, keeping the local CPU cores fully saturated with much lower overhead (shared memory vs. network passing).

8.4 Further improvements

While this study explores a range of optimization and parallelization techniques, the resulting implementation is still sub-optimal compared to established libraries such as OpenBLAS and NumPy. Throughout this work, several advanced optimizations were intentionally left unimplemented, including but not limited to:

- Fine-tuned cache blocking for L1, L2, and L3 caches (the current implementation relies on rough L1/L2 estimates).
- Register blocking to better exploit CPU registers and reduce memory traffic.
- Discovering alternative strategies for handling edge cases in Strassen's algorithm.
- Multi-level recursion distribution in the MPI implementation (currently only the first recursion level is distributed).

9 Conclusion

In this assignment, we successfully designed and parallelized an efficient version of Strassen's algorithm, using OpenMP for shared-memory task parallelism and MPI for distributed computing.

From a practical standpoint, our benchmarks revealed that despite Strassen's superior asymptotic complexity, the OpenBLAS library which utilizes a standard $O(n^3)$ algorithm, outperformed our implementation. For example, at $N = 10000$, OpenBLAS remained roughly $2.7\times$ faster than our best parallel implementation. This underscores that for the matrix sizes tested, the constant factors associated with memory management, recursion overhead, and communication (in the case of MPI) outweigh the theoretical reduction in floating point operations. OpenBLAS achieves this through aggressive low-level optimizations, such as hand-tuned assembly, register blocking, and SIMD instructions, which maximize the utilization of the hardware pipeline in ways that a high-level recursive implementation cannot easily match.

Throughout this assignment, we learned several lessons in parallel computing:

1. Big-O notation is not the sole determinant of speed. Constant factors driven by cache efficiency and instruction-level parallelism often dominate performance for practical input sizes.

2. The cost of spawning OpenMP tasks and transmitting data via MPI can quickly negate computational gains. Efficient parallel design requires minimizing this overhead, such as by limiting recursion depth or overlapping communication with computation.

3. Achieving peak performance requires deep awareness of the underlying hardware, including cache hierarchy and vector units, rather than solely relying on algorithmic improvements.

Acknowledgement

We would like to express our sincere gratitude to the HCMUT High Performance Computing Center (HPCC) for providing the computer cluster resources that facilitated the distributed memory experiments in this assignment.

References

[1] Brice Boyer et al. "Memory efficient scheduling of Strassen-Winograd's matrix multiplication algorithm". In: *arXiv preprint arXiv:0707.2347* (2009). URL: https://arxiv.org/abs/0707.2347.

[2] Sergey Slotin. *Matrix Multiplication*. Algorithmica. 2022. URL: https://en.algorithmica.org/hpc/algorithms/matmul/ (visited on 12/17/2025).